Justificación de decisiones de diseño tomadas - EcoHarmony Park

Introducción

Este documento justifica las decisiones de diseño tomadas en el momento de implementar la user story “Inscribirme a actividad” de EcoHarmony Park. La implementación se realizó aplicando el enfoque de Desarrollo Conducido por Pruebas (TDD), siguiendo el ciclo Red-Green-Refactor, permitiendo construir la funcionalidad a partir de pruebas automatizadas, incorporando de manera progresiva las reglas de negocios y mejorando el diseño de código sin comprometer el comportamiento previamente verificado.

La funcionalidad implementada contempla la selección de una actividad y un horario, la validación de los datos de los visitantes, el control de disponibilidad de cupos, la aceptación de los términos y condiciones correspondientes y la generación de una confirmación de inscripción con un código QR enviado por correo electrónico.

Justificación de arquitectura elegida

La arquitectura del sistema se estructuró en cuatro capas claramente delimitadas (Presentación, Aplicación/Controladores, Dominio y Verificación) para maximizar la testeabilidad, aislar las reglas de negocio y facilitar la aplicación rigurosa del ciclo de Desarrollo Conducido por Pruebas (TDD).

1. Capa de Dominio / Core de Negocio (El Foco del TDD)

- **Aislamiento Tecnológico:** Al diseñar la lógica de negocio en Python puro (Plain Old Python Objects) dentro de src/ecoharmony_park, sin dependencias de Flask ni de la web, se logró un entorno ideal para TDD. Las pruebas unitarias pueden ejecutarse de forma ultrarrápida directamente sobre el ServicioInscripcion y sus reglas (cupos, horarios, generación de QR) sin necesidad de levantar servicios externos.
- **Inmutabilidad para Pruebas Predecibles:** El uso de @dataclass(frozen=True) para definir modelos de datos como Visitante, SolicitudInscripcion y Confirmacion garantiza que el estado no mute inadvertidamente durante la ejecución. Esto asegura que las pruebas unitarias sean determinísticas, evitando efectos secundarios al momento de validar los casos de prueba de la cátedra.

2. Capa de Aplicación / Controladores (Adaptador REST)

- **Responsabilidad Única (Thin Controller):** El servidor Flask (app.py) actúa exclusivamente como un enrutador que intercepta peticiones HTTP/JSON hacia los endpoints específicos (/api/inscribir, /api/cupos, /api/config, /api/correos) y delega la ejecución al servicio de dominio.
- **Manejo Declarativo de Errores:** Al capturar las excepciones específicas de negocio lanzadas por el dominio y traducirlas a respuestas JSON con códigos HTTP (201, 400, 403), se facilita la verificación de los flujos de error.

3. Capa de Verificación (El Motor del Diseño)

- **Doble Nivel de Verificación:** La suite de pruebas en pytest (con 139 tests) dirige el diseño permitiendo dos vías de validación. Por un lado, verifica la lógica de negocio directamente instanciando las clases de la Capa de Dominio. Por otro, utiliza un TestClient para realizar pruebas de integración de caja negra sobre la Capa de Aplicación a través de HTTP/JSON.
- **Cobertura Exhaustiva:** Esta estructura permite cubrir la totalidad de las clases de equivalencia (CEV, CEI, CSV, CSI) exigidas, garantizando que cada iteración de refactorización no rompa la funcionalidad existente.

4. Capa de Presentación (Front-end Desacoplado)

- **Independencia Total:** Construida con index.html, style.css y app.js, esta capa maneja el estado visual local y se comunica de forma asíncrona (Fetch API) con el backend.
- **Validación de Casos de Uso:** La decisión de utilizar Vanilla JS y no incorporar frameworks pesados agiliza el desarrollo y permite la implementación de la barra de "Presets de Prueba" (CP1 a CP6). Esto complementa las pruebas automatizadas del backend brindando una herramienta visual para validar rápidamente los escenarios de aceptación desde el navegador.

Modelado

1. Modelado de Dominio con Tipos Fuertes e Inmutabilidad

- Se utilizaron estructuras inmutables mediante el decorador **@dataclass(frozen=True)** para las entidades principales como Visitante, TerminosActividad, SolicitudInscripcion y Confirmacion.
- Esta inmutabilidad garantiza que el estado de los objetos no sea alterado accidentalmente durante el flujo de inscripción.
- Se implementó la clase Actividad como una enumeración (Enum) de Strings, limitando los valores posibles a "Tirolesa", "Safari", "Palestra" y "Jardinería".

2. Jerarquía de Excepciones Personalizadas

- Se definió una clase base ErrorInscripcion de la cual heredan excepciones específicas de negocio, incluyen HorarioInvalido, SinCupo, DatosVisitanteInvalidos, EdadNoPermitida, TerminosNoAceptados e InscripcionDuplicada.
- El uso de estas excepciones granulares permitió diseñar los tests automatizados (fase *Red* del TDD) esperando errores exactos para cada caso de falla de los criterios de aceptación, como intentar inscribirse sin cupo o sin aceptar términos.

3. Centralización de Reglas de Negocio en la Capa de Servicio

- Toda la lógica y validación de reglas de negocio se centralizó en la clase ServicioInscripcion. Esta centralización fue producto directo de la fase *Refactor* del ciclo TDD: a medida que se escribía el código mínimo para hacer pasar las nuevas pruebas de usuario (fase *Green*), se identificó lógica de validación repetida. Para eliminar esta duplicación y mejorar la cohesión, el diseño nos llevó hacia el uso de constantes de clase que definen las reglas de negocio.
- Las reglas de negocio que se definieron como constantes de clase son HORA_APERTURA = 9, HORA_CIERRE = 17, ANTICIPACION_MINIMA, y la definición de ACTIVIDADES_CON_VESTIMENTA.
- El método validar_visitante consulta estas constantes para determinar dinámicamente si una actividad requiere talle de vestimenta o una edad mínima, en lugar de utilizar múltiples condicionales anidados.
- El control de capacidades se gestiona mediante un diccionario self.cupos, el cual descuenta la cantidad de personas solicitadas una vez validada la transacción en el método inscribir.

4. Desacoplamiento para la Generación de QR y Envío de Correos

- La funcionalidad de notificaciones se abstrajo en el método enviar_email, el cual funciona como una simulación local que guarda la confirmación en la lista self.correos_enviados.
- Este diseño evita la dependencia de un servidor SMTP real, permitiendo que las pruebas automatizadas verifiquen el envío inspeccionando la lista en memoria de forma instantánea.
- La generación de códigos QR se encapsuló en el método estático _crear_qr_uri, utilizando la librería reportlab para crear un gráfico SVG codificado en Base64.
- Este enfoque in-memory asegura que la validación del último criterio de aceptación (envío de email con QR) sea testeable sin necesidad de escribir archivos físicos en el disco.

5. Prevención de Inscripciones Duplicadas

- Se incorporó un diccionario self.inscripciones_existentes que mapea una tupla de DNI y horario hacia una actividad.
- El método _validar_solicitud normaliza el DNI del visitante y verifica contra este registro para lanzar la excepción InscripcionDuplicada si la persona ya posee una reserva en ese mismo horario.

Conclusión

La aplicación estricta del ciclo *Red-Green-Refactor* fue fundamental para el desarrollo de esta U.S. Pensar primero en las pruebas obligó a diseñar una arquitectura limpia que previno el acoplamiento temprano con el framework web (Flask) y dependencias externas como el envío de correos. Esto permitió que el diseño del software emergiera de forma natural hacia una solución modular, robusta y facilmente testeable. Finalmente, la cobertura exhaustiva de pruebas garantiza que el sistema cumple íntegramente con los criterios de aceptación y está preparado para futuras modificaciones sin riesgo de introducir regresiones.